

Design and Implementation of a Linguistic Wordle

Which linguistic features provide the most effective support for players when guessing a hidden target word in a Wordle-like game?

Project Report

Submitted by:

Authors: Jonas Patzke (3888339) and Veronika Rapp (4002536)

Email: Jonas.Patzke@uni-duesseldorf.de and Veronika.Rapp@uni-duesseldorf.de

GitHub: <https://github.com/Jonas-Patzke/linguistic-wordle>

Submitted to:

Supervisor: Rainer Osswald and Dr. Yulia Zinova

Course: Advanced Python for NLP (2026)

Date of Submission: October 5, 2026

Table of contents

1. Introduction
2. Project goal & Research question
3. Data & Tools
 - 3.1. Data
 - 3.2. Tools
4. Pipeline & Implementation
5. Quantitative and Qualitative Analysis
6. Challenges & Open Issues
7. Conclusion

Report

1. Introduction

In our era, mobile games are becoming more and more popular. One of the most popular games is wordle. A game, where you daily guess a mysterious five-letter word in six tries. And like every game, wordle has similar versions of itself, e.g.: guess the artist, guess the movie, guess the math equations and so on. But a version, where you have a special kind of hints, we personally never heard of it before. So, we made a linguistic-wordle, a wordle version specialized in linguistic aspects.

The main idea of the project is to create a wordle version where the guesses are supported by linguistic clues. So, you don't only have to rely on the position of the letter but have a meaningful clue like the word's part of speech, the syllables count, number of vowels and frequency of the missing word. This not only leads to a unique game experience but also lets you explore linguistic patterns.

The game is aimed at language enthusiasts, linguistics students, as well as developers who are interested in the spread of natural language (NLP). In this report we are going to talk about the project goal as well as research questions, how the project is set up, more precisely, which data and tools we use and the pipeline and implementation behind the project. After that, we'll take a look at the quantitative and qualitative analysis of the python project along with challenges and open issues we had to deal with.

2. Project goal & Research question

The main goal of the project is to analyze how linguistic information can assist players in word-search games. Since the original wordle only offers colors as hints, the game is very limited and simple. Linguistic features, on the other hand, can make the game more strategic and predictive.

The project is supposed to show the difference between a regular wordle game and a wordle game, where you have linguistic features supporting your guesses. With that we can find out how well linguistic features really help.

To illustrate the comparison, we investigate how the different clues influence the speed and accuracy of solving the puzzle. This allows us to answer our central research question: "Which linguistic features provide the most effective support for players when guessing a hidden target word in a Wordle-like game?". For that, we collect game results of different players who played the linguistic wordle and a second wordle without the linguistic hints, to evaluate the effectiveness of the clues. A closer look at the analysis of the game is going to happen in chapter 5. "Quantitative and Qualitative analysis".

3. Data & Tools

3.1. Data

For the project we used the license-free novel "Moby-Dick; or, The Whale" by Herman Melville from the website: www.gutenberg.org/ebooks/15. We cleaned the dataset with the help of some various functions, like `extract_chapters`, `clean_lowercase` and `tokenize`. Initially, we wanted to use the first three chapters of the novel and work with 5 – 10 letter words. Later we found it too difficult to guess a word with more than 7 characters. Therefore, we decided to take only 5 – 7 letter words. Since we did not want to lose our variety of words, we took the first 20 chapters of the novel for our dataset. After we extracted the chapters, we stored it in the file `extracted_chapters`, we began to clean the text by converting all the words to lowercase, tokenizing the text and lemmatizing the words. To make the words usable for our game, we removed stop words, punctuation, duplications and proper names.

Subsequently, when we were finished with processing the text, we analyzed the remaining words for their linguistic properties. For each word we have calculated the following features: number of letters, the part of speech, the frequency, the first letter of the missing word and number of syllables. To determine the linguistic properties of the words, we used various external resources and libraries. In order to count the syllables, we used the CMU Pronouncing Dictionary (CMUdict), which is available via the Python library NLTK. The CMUdict contains phonetic transcriptions of English words by which we can reliably determine the number of syllables. With the help of spaCy-Tagger we could determine the part of speech of a token. The number of vowels, the first letter of the word and the length of a word, we could easily calculate through the tokens themselves. This information was stored in a dictionary and subsequently saved as `propertie_dict.pkl`. This dictionary forms the basis for all linguistic analyses in the project and of course additionally for the hints in the game.

For the actual Wordle game we did not need all words, so we made a word list with only 5-7 letter words and saved them as `CMUdict_5_to_7.pkl` in our data file and used it as our guessable set for both wordle variants. We also created a second file "`CMUdict_5_to_7_without_propn.pkl`", like `CMUdict_5_to_7.pkl`, but without the proper names, to have a better comparison of the words POS-tag distribution between the words from the novel and the ones out of the CMU dictionary.

For our data analysis, we generated two separate Google-Spreadsheets by using the python library `gsread` with the combination of a google service account. The data is written on the table under program control and can be used later. This allows us to treat the spreadsheets like an external database and integrate the data directly into our analysis functions. For every Wordle game played, a new row is automatically created in a corresponding spreadsheet. The word being sought, the number of attempts, including the guesses made, whether the game was won, the duration of the game and the player are storage in both Spreadsheets. This allows us simple, platform-independent storage and direct access to the data during the analysis phase. Whoever collected data had to play both games equally often to ensure that we get qualitative data.

3.2. Tools

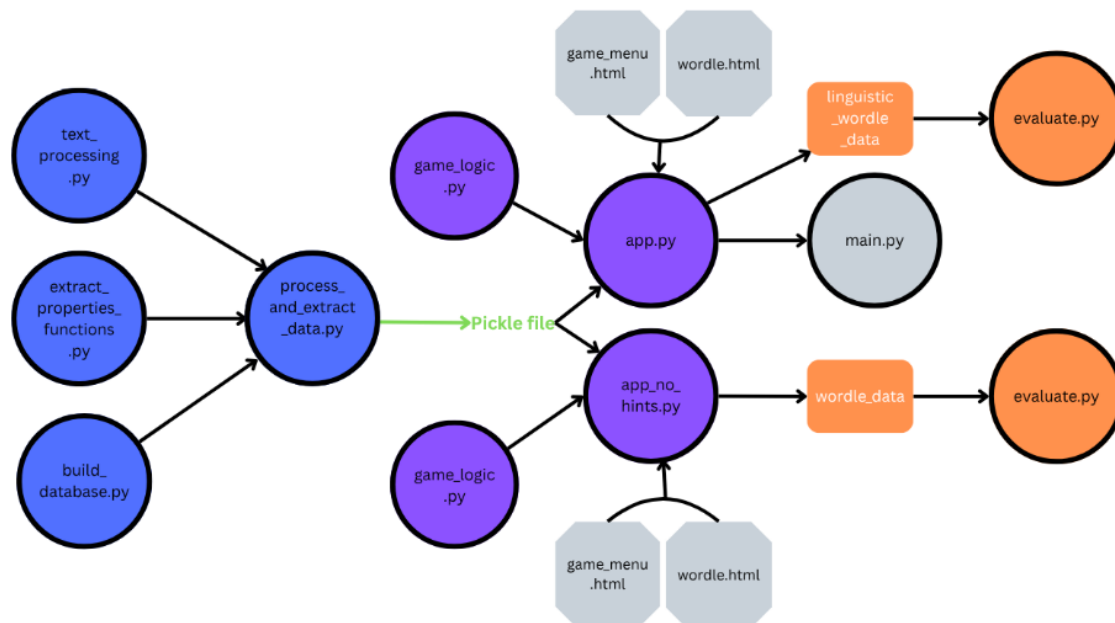
We used a wide variety of tools to implement the project and analyze the linguistic properties. For the extraction and analysis of the words we used NLTK (3.9.4), with which we could easily lemmatize, tokenize the words and identify further linguistic features of a word. You can find further details on text processing in section 3.1. "Data". Spacy (3.8.14), in our case spaCy-Tagger with the English language model `en_core_web_sm` (3.8.0), we needed for Determination of linguistic properties, as already mentioned in Chapter 3.1. "Data". The game data for both Wordle variants is stored in Google Sheets using `gsread` (6.2.1) and `google-auth` (2.58.1), further details on this can be found in section 3.1. "Data" too. To visualize the analysis results (e.g., frequencies, syllable distributions, vowel counts), we used `matplotlib` (3.11.2) to create simple, clear charts. To implement the Wordle game itself, we used Flask (3.1.3), which allowed us to create a simple web application that provides both the game mechanics and the hints.

We chose these tools because they provide reliable, well-established methods for linguistic analysis, data processing and visualization, which allowed us to build a reproducible and transparent workflow for the project.

4. Pipeline & Implementation

In this chapter, we describe the entire pipeline developed for the project's implementation. It encompasses all steps, ranging from data preparation and linguistic annotation to the implementation of the Wordle game and the subsequent statistical analysis. The pipeline is modular designed, ensuring that each sub-process is clearly delineated and can be further developed and adapted independently. The project includes a README file that documents the

project structure, the modules used, and the key processing steps. Overall, it consists of four key phases: data preparation and property determination, implementation of the Wordle game, user interface (frontend / HTML templates) and collection and analysis of the game data.



Our first step of the pipeline was to extract and process the data, to make the data suitable for the game. In the module `text_processing.py` our project has started. The module contains the basic functions for extracting the chapters, text cleaning, such as tokenization, lemmatization, and the removal of unnecessary words, like already in chapter 3.1. “Data” described in detail. `Text_processing.py` contains the text-cleaning components, creates the file `extracted_chapters` with the first 20 chapters of the novel and is therefore a functional module. The module `process_and_extract_data.py`, on the other hand, is responsible for executing the entire data processing process. It calls the functions from `text_processing.py` and executes them in the correct order to generate the final dataset. The module loads and cleans the text step by step, implements processing steps sequentially, extracts linguistic features, generates property dictionaries, saves a word list and thus creates the final datasets. The three dictionaries that emerge from this module with the help of the module `build_database.py` are `propertie_dict.pkl`, `CMUdict_5_to_7.pkl` and `CMUdict_5_to_7_without_propn.pkl`. Like mentioned in chapter 3.1. “Data”, the `propertie_dict.pkl` is the basis for all linguistic analyses, `CMUdict_5_to_7.pkl` is used for the wordle game and `CMUdict_5_to_7_without_propn.pkl` was created for the comparison. `Build_database.py` provides only functions for creating and saving dictionaries. It is used exclusively by the pipeline in `process_and_extract_data.py`. After we finished extracting all required words, we examined the linguistic properties of the words in the module `extract_properties_functions`. The module is closely linked to `process_and_extract_data.py`, as it provides the actual linguistic analysis. There we compute the length, the part of speech, the frequency and the syllables of the words. Further details on which tools we used to calculate the properties, you can find in chapter 3.1. “Data”.

As second phase we had the implementation of the Wordle game. The complete Wordle game mechanics are implemented here. In the module `game_logic.py` is where the random word for the game is selected from the `CMUdict_5_to_7_without_propn.pkl` with the help of the built-in library module `random`. In this module the word is also checked for validity, if the guess is correct, feedback on correct letter positions and the processing of the clues. `Game_logic.py` forms the functional basis for both Wordle variants. The modules `app.py` and `app_no_hints.py` are identical. The only thing which distinguishes the two is that in the module `app_no_hints.py` we changed every hint to an empty string and selected a different port to open the game. Therefore we have two

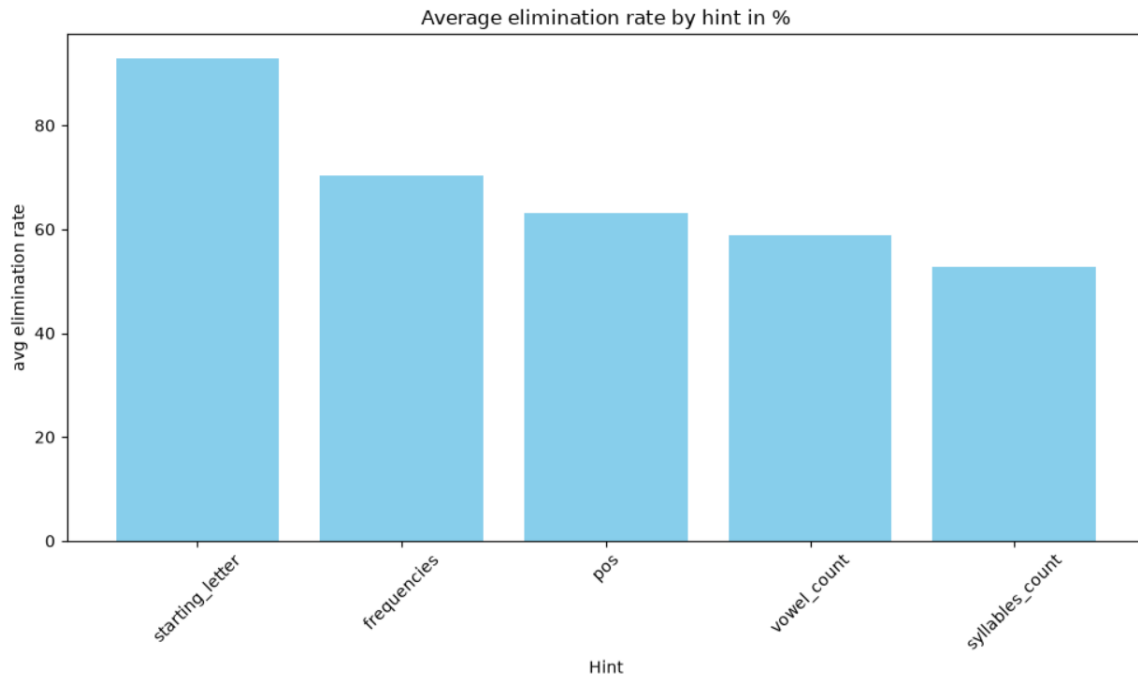
different Wordle variants. The modules form the backend of the web application and connect the game mechanics from `game_logic.py` with the graphical user interface from the HTML templates. In addition, both modules have a function that writes the game data to the corresponding Google Spreadsheet after every game. The tools we used to do this are explained in greater detail in chapter 3.1. “Data” and 3.2. “Tools”. All criteria that are needed for the analysis, like duration of the game and try number entered the spreadsheets for the analysis are defined in `app.py` and `app_no_hint.py`. The `credentials.json` file contains the login data for the Google service account and is thus required by `app.py` and `app_no_hints.py`. The file enables the game data to be written to the two spreadsheets, in other words, without this file, the modules would not be able to save data. The `player.txt` file is read in by the modules. It contains the player’s name and is transferred to the game data to the spreadsheet. These files are used exclusively for game management.

The third phase is the user interface (frontend / HTML templates). This phase encompasses the entire user interface of the Wordle game. While the game mechanics are implemented in Python modules, HTML templates are used to facilitate player interaction within the browser. The `game_menu.html` module serves as the main menu, allowing players to choose between the two Wordle variants. The `wordle.html` module represents the actual game interface, visually displaying the input fields, feedback on guessed words, and in the linguistic variant the hints. The HTML templates communicate with the backend modules `app.py` and `app_no_hints.py` via Flask. Each player input is sent via a form to the corresponding Python script, processed there, and subsequently returned to the HTML interface. The module `main.py` serves as the startup script for the web app. When executed, it opens the browser, loads the HTML interface, and starts the Flask server that enables communication between the frontend and backend. Thus, `main.py` is a central component of the user interface, as it makes the HTML layer technically accessible. The phase constitutes the visual and interactive layer of the pipeline that makes the Wordle game accessible to users.

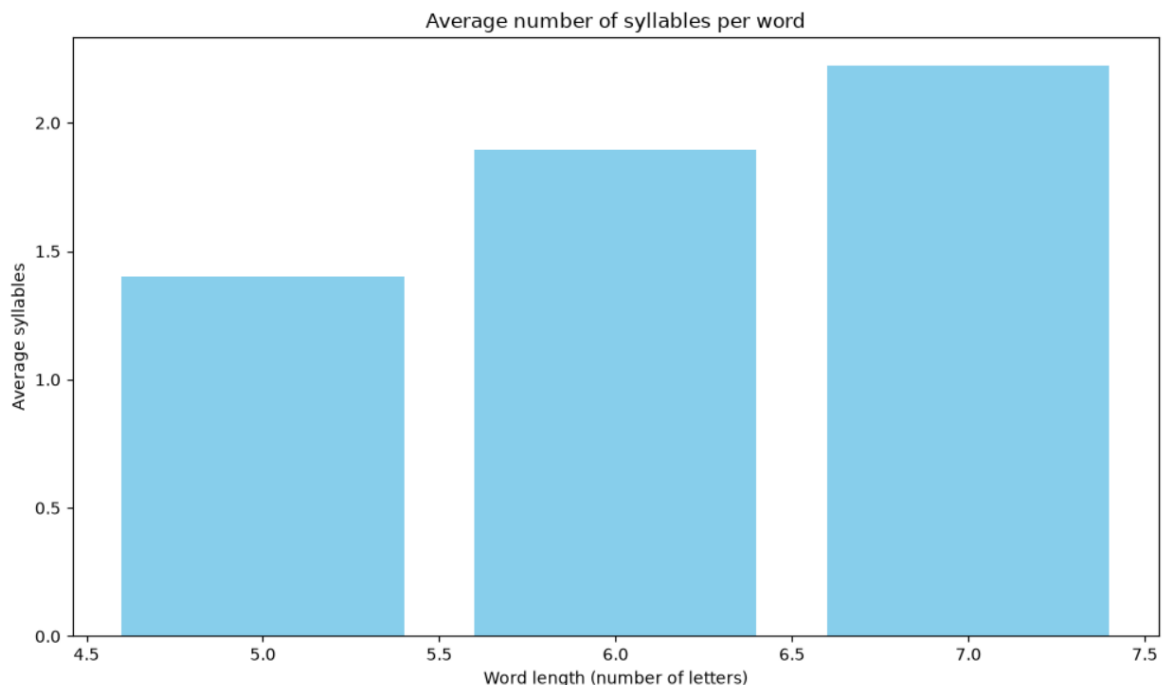
The last phase of our pipeline is the collection and analysis of the game data. It encompasses the collection, analysis, and visualization of game data from both Wordle variants. The two tables, `linguistic_wordle_data` and `wordle_data`, serve as databases storing information for each word played, such as the number of attempts, won or not, playtime, player name, and the words, which the players provided as guesses. This data is subsequently loaded into the module `evaluate.py` with the help of the `credentials.json` and through various functions statistically analyzed. The functions analyze two focal points: the game data and the words we use for the game. For the game data analysis, we created with the module tables, which shows us the average tries by frequency, part of speech, syllables and vowels as well as their win rate. For an overview we constructed an analysis file with all the aspects mentioned. These files, we saved as csv in an extra file `tables_and_charts` in data. For the word analysis we created also in `evaluate.py` charts in .png format the average syllables by length, average vowels by length, the top 10 common words, the top 5 common first letters and the average elimination rate. You can also find the png files in `tables_and_charts`. The results are presented visually to make the differences between the classic and linguistic versions of Wordle clearly apparent. However for us the csv tables looked the best in GitHub and looked like unsorted data in Excel, you may consider it.

5. Quantitative and Qualitative analysis

To answer the research question regarding which linguistic properties influence game performance in a Wordle-like game and which clues offer the greatest benefit, the collected data are analyzed below using both quantitative and qualitative methods. While the quantitative results reveal statistical correlations between linguistic features and game performance, the qualitative analysis allows for a contextual interpretation of these patterns.

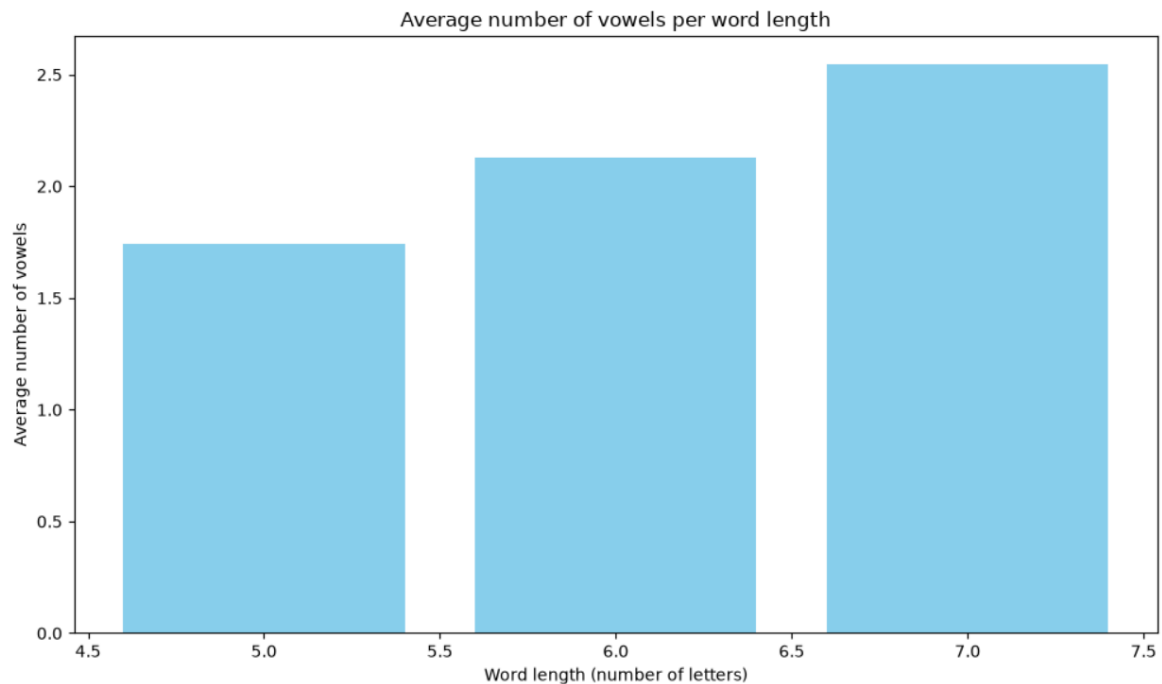


The average elimination rate by hint ranges between approximately 55% and 90% for the different hints. To get this chart, we calculated the expected value. The initial letter eliminates most words with ca. 90% because it is a very strong structural feature: once the first letter is determined, a large portion of the word list is ruled out. Frequencies with ca. 70% and POS tags with ca. 65% are moderately strong clues, as they represent statistical rather than structural properties. Vowel with ca. 60% and syllable counts with only ca. 55% are weaker hints because many words share similar values, making these features less discriminating.

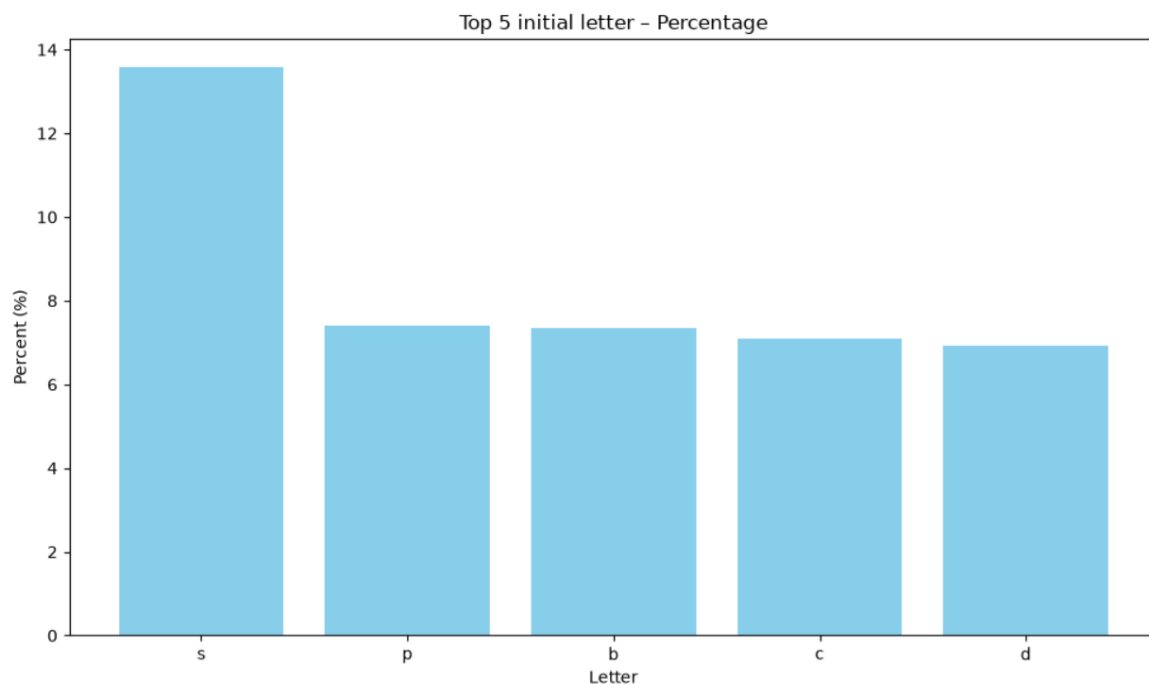


The average number of syllables increases with word length, since longer words typically contain more syllables, a result of the natural structure of English. A 5-letter word has on average ca. 1.4 syllables, a 6-letter word ca. 2 syllables and a 7-letter word even ca. 2.2 syllables. For the game, this means that words with more syllables often appear more complex,

even though they are not necessarily harder to guess. The number of syllables influences the perception of difficulty rather than the actual success rate.

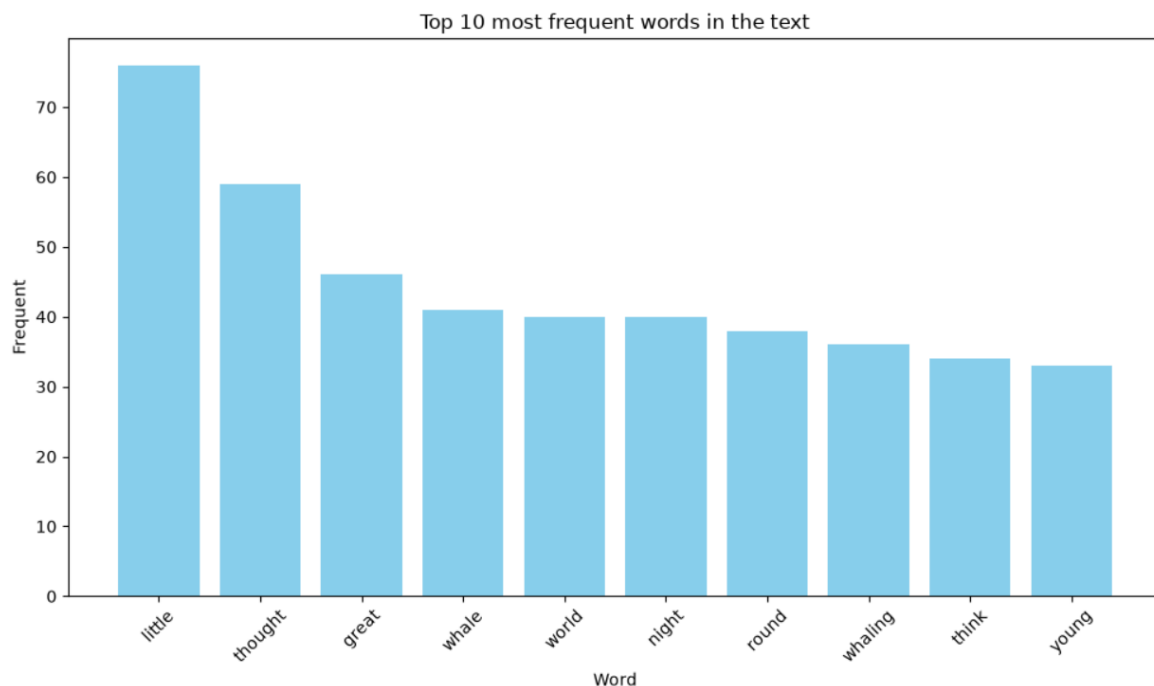


Like in average numbers of syllables, the average number of vowels increases with the word length. A 5-letter word has on average ca. 1.7 vowels, a 6-letter word already ca. 2.1 vowels and a 7-letter word ca. 2.6 vowels. More letters mean more room for vowels. Interestingly, words with many vowels are often easier to guess because vowels appear more frequently in English and players test them early on. This explains the higher success rates observed later for words with 4–5 vowels.



This chart shows that the most common initial letter is “s” with ca. 14%. “p”, “b”, “c” and “d” are also in the top 5 with ca. 7% each. This means “s” appears about twice as frequently as the other four letters and is an extremely productive initial letter in English (plural forms, verbs, common nouns). For the game, this means that words starting with the letter “s” are statistically

more frequent. In Fact, players intuitively exploit many starting words like "start," "sailor," "strange," etc. The other four letters are also frequent, but significantly less dominant.



The ten most frequent words range from ca. 30 to 70 appearances. The most frequent word is “little” with a little more than 70. “thought” is in the second place with ca. 60 appearances. “great” follows with ca. 46, then comes “whale”, “world” and “night” with around 40 times each. “round” it at ca. 38, “whaling” at 36, “think” at 35 and “young” is in the last placed with around 34. These frequencies reflect the stylistic structure of the source text. For the game, this means that frequent words are often easier to guess because they are more strongly anchored in the players' mental lexicons. Rarer words such as "whaling" or "young" can be more difficult, despite their brevity.

	avg_tries_by_pos	avg_tries_by_syllables	avg_tries_by_vowels	avg_tries_by_frequency	winrate_by_pos	winrate_by_syllables	winrate_by_vowels	winrate_by_frequency
ADV	4.6				1.0			
VERB	4.6				0.8			
NOUN	5.08				0.8			
ADJ	5.2				0.7			
3.0		4.75	5.153846153846154	5.6		1.0	1.0	0.6
1.0		4.928571428571429	4.25	4.851851851851852		0.6428571428571429	1.0	0.8148148148148148
2.0		5.035714285714286	4.96969696969697	5.285714285714286		0.8214285714285714	0.696969696969697	0.5714285714285714
0								
14.0				3.0				1.0
6.0				4.0				1.0
4.0				4.333333333333333				1.0
5.0				5.0				1.0
36.0				5.0				1.0
7.0				5.5				1.0
17.0				6.0				1.0

In the category part of speech, adverbs and verbs have a higher success rate with only 4.6 tries in game and 100% and 80% win rate. Nouns and adjectives, on the other hand, have ca. 5.1

average tries each. Nouns have with 80% a higher win rate than adjectives with only 70%-win rate. Verbs and adverbs are easier to guess due to their semantically clearer and more frequent form. Nouns and adjectives are more diverse and less predictable, which lowers the solve rate. The success rate for words with 3 syllables are the highest with 4.75 tries and 100%-win rate. Words with 1 and 2 syllables need in average 5 tries and with one syllable the win rate is at only 64% and for 2 syllables 82%. Three-syllable words are surprisingly easy, presumably because they often feature distinct vowel patterns. One-syllable words, on the other hand, are harder, as there are many possible candidates (e.g., "think," "drink," "blink"). The average try number for words with 3 and 2 vowels is at around 5 tries, for 1 vowel at 4.25 and the win rate for one and three vowels is at 100%. The win rate for words with 2 vowels lies at 70%. Words with many vowels are easier due to the early testing of vowels by the player, thereby quickly gaining insight into the word's structure. The frequency of a word shows the higher the frequency, the higher the statistical win rate, since words with a higher frequency as 3 have always a 100%-win rate. The try number in this category varies greatly and ranges between 3 and 6 tries. The values show that words with very low frequency (e.g., lower than 3.0) have a significantly lower success rate. In contrast, words with medium or high frequency are almost always solved.

We have also compared both spreadsheets `linguistic_wordle_data`, the data of the linguistic wordle and `wordle_data`, the data of the original wordle. There you can see the differences very clearly. For the wordle with linguistic hints is the average number of attempts at 4.96 and in the normal wordle at around 5.38. The win rate also differs massively. For the linguistic wordle the win rate is at 80% and for the original wordle at only 56%. However, the duration time differs unexpectedly. The linguistic wordle has a duration time of a game at ca. 280.66 seconds and the other wordle at ca. 254.48 seconds. The version with hints leads to fewer attempts and a higher success rate, which shows that players arrive at the solution with fewer tries and win significantly more often. The version without hints requires more attempts and shows a significantly lower success rate, but timewise its faster. That means without linguistic support, the game is noticeably more difficult. The time shows surprisingly that the players are sitting at the linguistic wordle longer. It could represent that the player thinks it over and has more information to think about instead of simply excluding letters. The time may show such a result, due to deeper thinking, if you have more information, and more trying to get painted letters, if you have less information. The less information leads to some fast guessed words to get first hints and eventually the player is unlucky with the fast hints and gains just little information.

Overall, the combined analysis shows that linguistic properties such as the initial letter, word frequency, part of speech, and vowel count have a measurable influence on the probability of solving the puzzle. The research question can thus be clearly answered: structural and frequency-based linguistic features provide the greatest support to players, and the Wordle variant with hints leads to significantly better game performance than the classic version.

6. Challenges & open issues

Challenges

Proper Names

Since the start of the project, we decided to remove proper names from the text and did it by filtering all the words that got the POS-tag "PROPN". Later we wanted to compare the POS-tag distribution between our data and the words from the CMU dictionary, therefore we also deleted proper names out of it. As soon as the game was playable, we realised that there are some words that just were not guessable, e.g. "Dolphin" or "Elefant". For plenty of time we thought the words just were not part of the CMU dictionary, but through some testing we found out otherwise. After some thinking we remembered that we deleted the proper names from the CMU dictionary and tested if the words "Dolphin" and "Elefant" are tagged as proper names. The testing showed that they were. Due to that we created two pickle files for the words in the CMU dictionary that

are between 5 to 7 letters long. One with and one without proper names and now we got nearly 60,000 guessable words.

Google Spread sheets instead of Microsoft Excel

Our first thought was to store the data we collect from the games in excel spreadsheets or in pandas' data frames. Further investigations turned out that if we would do it that way, we would not be able to play the game at the same time and collect data. We would have to push these gained data on GitHub and through that we would eventually replace the gained data of the other. Through quick research we found out about Google spreadsheets and that there is a way to get a connection to our python code.

Evaluating the data

Although our statistical analysis showed that some hints were theoretically more informative than others, we had no evidence that they improve player performances. Therefore, we faced the challenge of validating whether the provided hints had a measurable impact on gameplay. We decided to make another game and open another Google spreadsheet and compare the data. To prevent coding everything from scratch, but without hints, we took the code with hints, changed the hints into empty strings and connected the other spreadsheet. The idea is to play both games on the same conditions and compare the data of both spreadsheets to see if there are significant differences.

Open Issues

Morphemes

Initially, we planned to use morphemes as hints, after lemmatization we found out that most English words in our dataset consist of only one morpheme. As a result, morpheme-based hints would have provided little additional information to the player.

We therefore considered introducing an additional game mode, the No-Lemma Mode, in which words would not be lemmatized. Unfortunately, due to time constraints, we were unable to implement this feature.

This additional game mode could provide an interesting basis for future work. It would allow further investigations, like how lemmatized and non-lemmatized words differ in the aspect of other linguistic properties.

Proper Names

Due to the challenge encountered with proper names, it is obvious that there also words from the novel get labelled as "PROPN", even though they could be classified as "NOUN". As a result, some words that might have been suitable solution words were excluded from the game.

One idea was to compare the filtered proper names with the entries from the CMU dictionary and create the intersection of both sets. This approach could eventually recover words that were incorrectly classified as proper names. Due to missing time, we were unable to implement and evaluate this approach. We are not sure if all intersecting words are nouns.

6. Conclusion

The central research question of this study was whether linguistic properties could provide measurable assistance to players in a Wordle-style game and which of these features had the greatest impact on the likelihood of solving the puzzle. To address this question, a comprehensive linguistic data pipeline was developed to extract words from a literary text, annotate them linguistically, and integrate them into two versions of a Wordle-style game: a classic version and a version featuring linguistic hints.

The results clearly demonstrate that linguistic information not only alters the game but also noticeably improves player performance. Hints such as the initial letter, word frequency, or part of speech provide players with structural guidance, narrow down the pool of potential candidates, and facilitate the process of finding the solution. The combination of structural and frequency-based features proved particularly effective. Thus, the study confirms that linguistic properties are not merely of theoretical interest but offer tangible benefits in a practical application context, such as a word game.

Furthermore, the comparison between the two game versions reveals that the integration of linguistic hints has a lasting impact on the player experience: the version with hints leads to higher success rates, fewer attempts, and a generally more efficient style of play. This underscores the fact that linguistic features constitute a valuable tool for designing interactive applications and that their targeted use can enhance usability and playability.

Overall, the research question can be answered unequivocally: structural and frequency-based linguistic properties provide the greatest support to players, and the integration of such hints significantly improves performance. The results indicate that linguistic features constitute a valuable tool for word game design and that their targeted use can have a lasting impact on the player experience.